

CSCI 232

232 Algorithms & Data Structures

LECTURE: 13

J. L. Pach

TODAY'S AGENDA

1. Why This Matters
2. Structs in C — Data Only
3. Function Pointers — Behaviors
4. Combining Structs + Function Pointers — Birth of Objects
5. Quasi-Constructors in C
6. Full Example: Point Object in C
7. Comparison: C vs C++
8. Advanced Patterns

FROM C TO C++

Objects Born from Structs

UNDERSTANDING OOP THROUGH LOW-LEVEL C IMPLEMENTATION

1. WHY THIS MATTERS 1/2

The Problem Students Face

Most students learn `C`, then jump to `C++`, and suddenly... objects exist.

They don't understand:

- ⚒ Where did methods come from?
- ⚒ Why do I need `this`?
- ⚒ What's really happening under the hood?
- ⚒ Why does `C++` "look different" if it's built on C?

The Solution: Show the Evolution

1. WHY THIS MATTERS 2/2

By building objects in `C`, students see:

- Objects are just data + function pointers
- Methods are just functions that know about their data
- `this` is just a hidden first parameter
- `C++` is syntactic sugar on top of `C` patterns

This builds REAL understanding, not cargo cult programming.

2. Structs in C — Data Only

BASIC STRUCTURE

```c
#include <stdio.h>
#include <math.h>
// Simple Point struct - data only
struct Point {
    int x;
    int y;
};
// Free function (C-style)
double distance_from_origin(struct Point* p) {
    return sqrt(p->x * p->x + p->y * p->y);
}
int main() {
    struct Point p;
    p.x = 3;
    p.y = 4;
    printf("Point: (%d, %d)\n", p.x, p.y);
    // Usage:
    double dist = distance_from_origin(&p);
    printf("Distance: %.2f\n", dist);
    return 0;
}
```

- Works, but ...the function is disconnected from the data.
- **Problem**: No Behavior
- What if we want to calculate the distance from origin?
- Store the function inside the struct (OOP-style)
- That's what we're going to do.

3. Function Pointers — Behaviors

- A function pointer is a variable that stores the address of a function.

```c
// Define a function
int add(int a, int b) {
    return a + b;
}
// Create a function pointer
int (*operation)(int, int);
// Point it to the function
operation = &add;
// Call through the pointer
int result = operation(3, 5);
printf("Result: %d\n", result);
// Output: Result: 8
```

> Notice: We pass &calc as first parameter (like "this" in C++)

- Function Pointers in Structs
  We can store function pointers as struct members:

```c
struct Calculator {
    int x;
    int y; // Function pointer for addition operation
    int (*add)(struct Calculator* self);
};
// Implementation of add function
int calc_add(struct Calculator* self) {
    return self->x + self->y;
}
int main() {
    struct Calculator calc;
    calc.x = 3; calc.y = 5;
    calc.add = &calc_add;
    int result = calc.add(&calc); // Call the "method"
    printf("Result: %d\n", result);
    return 0;
```

4. Combining Structs + Function Pointers — Birth of Objects 1/2

The Concept -> An object = data + methods (function pointers)

```c
// Step 1: Define interface (what methods exist)
struct Shape {
    double width;
    double height;
    // Methods (function pointers)
    double (*get_area)(struct Shape* self);
    double (*get_perimeter)(struct Shape* self);
    void (*print)(struct Shape* self);
};
// Step 2: Implement methods for Rectangle
double rect_get_area(struct Shape* self) {
    return self->width * self->height;
}
double rect_get_perimeter(struct Shape* self) {
    return 2 * (self->width + self->height);
}
void rect_print(struct Shape* self) {
    printf("Rectangle: %f x %f\n", self->width, self->height);
}
```

```c
// Step 3: Create an instance
int main() {
    struct Shape rect;
    rect.width = 5.0;
    rect.height = 3.0;
    // Attach methods to the instance
    rect.get_area = &rect_get_area;
    rect.get_perimeter = &rect_get_perimeter;
    rect.print = &rect_print;
    // Use the object
    rect.print(&rect);
    printf("Area: %.2f\n", rect.get_area(&rect));
    printf("Perimeter: %.2f\n", rect.get_perimeter(&rect));
    return 0;
}
```

Output:

Rectangle: 5.000000 x 3.000000

4. Combining Structs + Function Pointers — Birth of Objects 2/2

WHAT JUST HAPPENED?

- We created an object with data (width, height)
- We added behaviors (methods) to it
- We called those behaviors on the object
- Each method received the object as first parameter (self/this)

> This is literally how objects work.

The Problem with Manual Initialization

```c
struct Point {
    int x;
    int y;
    double (*distance)(struct Point* self);
};
double point_distance(struct Point* self) {
    return sqrt(self->x * self->x + self->y * self->y);
}
```

```c
int main() {
    struct Point p;
    p.x = 3;
    p.y = 4;
    p.distance = &point_distance;  // Manual! Easy to forget!
    printf("Distance: %.2f\n", p.distance(&p));
    return 0;
}
```

```
Output:
Point(3, 4)
Distance from origin: 5.00
```

- **Problem**: Easy to forget to attach methods. Error-prone.
- **Solution**: Initialization Function (Quasi-Constructor)

5b. Quasi-Constructor and Quasi-Destructor in C

⚒ For objects that allocate memory:

```c
#include <stdlib.h>
#include <string.h>
struct String {
    char* data;
    int length;
    void (*print)(struct String* self);
    void (*destroy)(struct String* self);
};
void string_print(struct String* self) {
    printf("String: %s (length: %d)\n", self->data, self->length);
}

void string_destroy(struct String* self) {
    if (self->data != NULL) {
        free(self->data);
        self->data = NULL;
    }
    self->length = 0;
}
```

```c
// Constructor
struct String String_create(const char* text) {
    struct String s;
    s.length = strlen(text);
    s.data = (char*)malloc(s.length + 1);
    strcpy(s.data, text);

    s.print = &string_print;
    s.destroy = &string_destroy;

    return s;
}

int main() {
    struct String str = String_create("Hello, World!");
    str.print(&str);

    // Must manually call destructor
    str.destroy(&str);
    return 0;
}
```

6. Full Example: Point Object in C

COMPLETE, PRODUCTION-STYLE IMPLEMENTATION

```c
// File: point.h
#ifndef POINT_H
#define POINT_H
typedef struct Point {
    int x;
    int y;
    // Method pointers
    double (*distance_from_origin)(struct Point* self);
    double (*distance_to)(struct Point* self, struct Point* other);
    void (*print)(struct Point* self);
    void (*translate)(struct Point* self, int dx, int dy);
} Point;
// Constructor
Point Point_create(int x, int y);
#endif
```

```c
// File: point.c
#include "point.h"
#include <stdio.h>
#include <math.h>
// Method implementations
double point_distance_from_origin(Point* self) {
    return sqrt(self->x * self->x + self->y * self->y);
}
double point_distance_to(Point* self, Point* other) {
    int dx = self->x - other->x;
    int dy = self->y - other->y;
    return sqrt(dx * dx + dy * dy);
}
void point_print(Point* self) {
    printf("Point(%d, %d)\n", self->x, self->y);
}
void point_translate(Point* self, int dx, int dy) {
    self->x += dx;
    self->y += dy;
}
// Constructor
Point Point_create(int x, int y) {
    Point p;
    p.x = x;
    p.y = y;
    // Bind methods
    p.distance_from_origin = &point_distance_from_origin;
    p.distance_to = &point_distance_to;
    p.print = &point_print;
    p.translate = &point_translate;
    return p;
}
```

6. FULL EXAMPLE: POINT OBJECT IN C

```c
// File: main.c
#include "point.h"
#include <stdio.h>
int main() {
    // Create two points
    Point p1 = Point_create(3, 4);
    Point p2 = Point_create(0, 0);
    // Use methods
    printf("p1: ");
    p1.print(&p1);
    printf("Distance from origin: %.2f\n",
           p1.distance_from_origin(&p1));
    printf("Distance between p1 and p2: %.2f\n",
           p1.distance_to(&p1, &p2));
    // Modify object
    p1.translate(&p1, 2, 3);
    printf("After translation: ");
    p1.print(&p1);
    return 0;
}
```

Output:

p1: Point(3, 4)

Distance from origin: 5.00

Distance between p1 and p2: 5.00

After translation: Point(5, 7)

KEY INSIGHTS FOR STUDENTS

1. Objects are Just Data + Function Pointers

```
Object = { data, method1_ptr, method2_ptr, ... }
```

Nothing magical.

2. Methods are Functions with Hidden First Parameter

In C: explicit

```
p.distance(&p);
```

In C++: implicit `this`

```
p.distance();
```

3. Constructors are Just Functions

They initialize the object and bind the methods.

4. Classes are Just Fancy Structs

Common Mistakes to Avoid

Mistake 1: Forgetting to Pass self

```
// WRONG:
double dist = p.distance();
// RIGHT:
double dist = p.distance(&p);
```

> This is why C++ hides it—easier to forget.

Mistake 2: Not Initializing Method Pointers

```
//WRONG:
struct Point p;
p.x = 3;
p.y = 4;
p.distance(&p);  // Crash! distance is uninitialized
// RIGHT:
struct Point p = Point_create(3, 4);
p.distance(&p);  // Works
```

Mistake 3: Memory Leaks in "Destructors"

```
// WRONG - forgot to destroy:
struct String s = String_create("Hello");
s.print(&s);
// Memory leak!
// RIGHT:
struct String s = String_create("Hello");
s.print(&s);
s.destroy(&s);  // Must explicitly destroy
```

> This is why C++ has RAII (automatic destructors).

CONNECTION TO C++

What C++ Does Automatically

1. Method Binding: Automatically binds methods to struct
2. This Pointer: Automatically passes "this"
3. Constructor Calling: Automatically calls constructor
4. Destructor Calling: Automatically calls destructor
5. Method Resolution: Automatically finds right method

Why Understanding C Version Matters

When C++ acts weird, you can debug by thinking:

⚒ "What struct data is this?"

⚒ "What function pointer got called?"

Summary

THE EVOLUTION

- C Struct:
  `struct Point { int x; int y; }`
  Just data, no behavior
- Function Pointer:
  `void (*print)(struct Point* self);`
  Now we can add behavior
- Quasi-Object:
  `struct Point { x, y, print_ptr, ... }`
  Data + methods, like an object
- Constructor:
  `Point_create() { bind methods }`
  Automatic initialization
- C Object System:
  Production-ready manual objects
- C++ Class:
  `class Point { ... }`
  Same thing, hidden details
- Other Languages:
  Python, Java, C#, etc.
  All evolved from this pattern

All modern OOP languages are variations on this theme.

THANK

You